

UNIT -2

Data Link Layer and Medium Access Sub Layer: Error Detection and Error Correction - Fundamentals, Block coding, Hamming Distance, CRC.

2.1 Data link Layer and Medium access sub layer: Error Detection techniques:

What is the data link layer?

The data link layer is the protocol layer in a program that handles the moving of data into and out of a physical link in a network. The data link layer is Layer 2 in the Open Systems Interconnection ([OSI](#)) architecture model for a set of telecommunication protocols.

Data bits are encoded, decoded and organized in the data link layer, before they are transported as [frames](#) between two adjacent nodes on the same LAN or WAN. The data link layer also determines how devices recover from collisions that may occur when nodes attempt to send frames at the same time.

The data link layer has two sublayers: the logical link control (LLC) sublayer and the media access control ([MAC](#)) sublayer.

As described by the IEEE 802 LAN specification, the role of the LLC sublayer is to control data flow among various applications and services, as well as provide acknowledgement and error notification mechanisms. The LLC sublayer can then talk to a number of IEEE 802 MAC sublayers, which control access to the physical media for transport. It is also responsible for the physical addressing of frames.

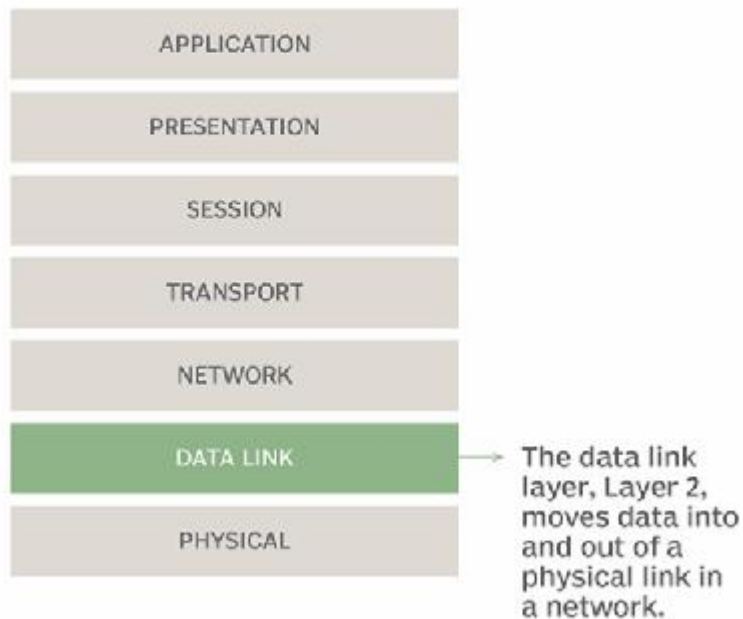
Two common MAC layer types include [Ethernet](#) and [802.11](#) wireless specifications.

Functions of the data link layer

The data link layer has three main functions:

- It handles problems that occur as a result of bit transmission errors.
- It ensures data flows at a pace that doesn't overwhelm sending and receiving devices.
- It permits the transmission of data to Layer 3, the [network layer](#), where it is addressed and routed.

The OSI model: Data link



The data link layer in the OSI model

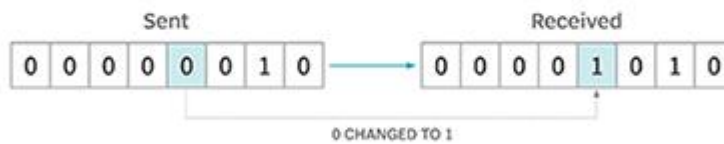
Data link layer and error detection

The data link layer ensures an initial connection has been set up, divides output data into data frames and handles the acknowledgements from a receiver that the data arrived successfully. It also ensures incoming data has been received successfully by analyzing bit patterns at special places in the frames.

If an error occurs, the data link layer notifies higher-level protocols that something has happened to the physical link. Frame sequencing capabilities within the data link layer permit the receiving device to reorder frames that might have been transmitted out of sequence. The data link layer verifies the packet is unimpaired.

The data link layer also manages flows by enabling devices on a link to detect congestion. Nearby devices then transmit congestion information, so traffic can be rerouted accordingly.

Bit transmission error detected by data link layer



Introduction:

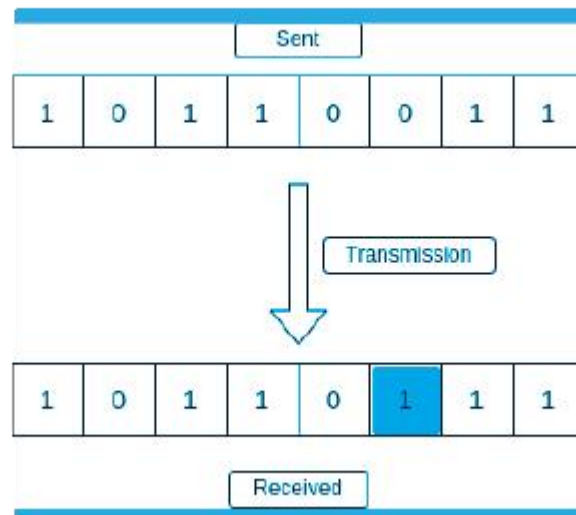
How do Errors Occur in Data Transmission?

Errors in data transmission can occur for a variety of reasons, such as electromagnetic interference, noise, faulty hardware, or even software glitches. These errors can lead to the alteration, insertion, deletion, or inversion of bits within the transmitted data, thereby compromising the accuracy and authenticity of the information. Given the critical nature of many network applications, it's imperative to employ error detection techniques to identify and rectify these errors.

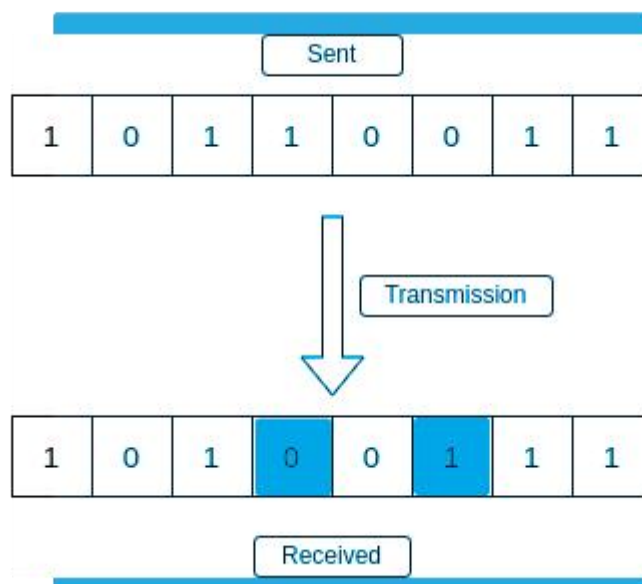
Types of Errors

In the context of computer networks and data transmission, errors can occur due to various factors, leading to the corruption or loss of data. These errors can be categorized into different types based on their origins and characteristics. Here are some of the most common types of errors in computer networks:

1. Single-Bit Error: A single-bit error occurs when one bit in a data unit changes from 0 to 1 or from 1 to 0 due to noise, interference, or other factors. Single-bit errors are relatively common and can usually be detected and corrected using error detection techniques like parity checks or Hamming codes.



2. Burst Error: A burst error involves the consecutive alteration of multiple bits within a data unit. These errors can occur due to factors like signal attenuation, electrical interference, or synchronization issues. Burst errors can be more challenging to detect and correct than single-bit errors, especially if they exceed the capabilities of the error detection mechanism in use.



3. Random (Transient) Error: Random errors are temporary and unpredictable changes in the data due to factors like temporary electrical interference, cosmic rays, or momentary glitches. These errors are typically short-lived and may not always be detectable, depending on the error detection mechanism employed.

4. Systematic Error: Systematic errors, also known as deterministic errors, are consistent and repeatable errors that occur due to flaws in the system or equipment. These errors can arise from faulty hardware components, misconfigurations, or software bugs. Systematic errors can have a significant impact on data integrity and may require thorough troubleshooting and correction.

5. Noise: noise refers to any unwanted signal or interference that distorts the original data. Noise can be introduced during data transmission due to various sources, such as electromagnetic interference, cross-talk, or channel attenuation. While noise may not always lead to explicit errors, it can degrade the quality of the transmitted data and affect the overall reliability of the communication.

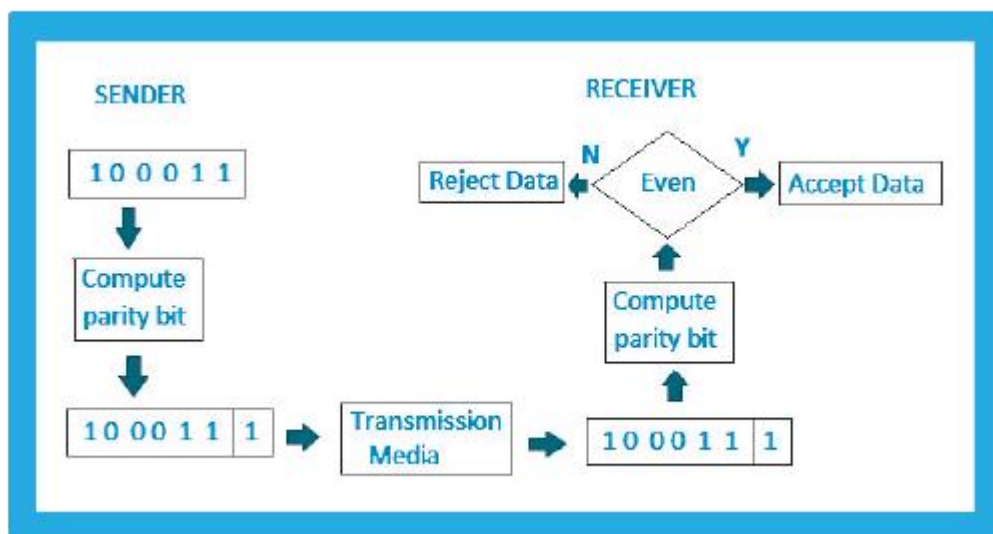
Importance of Error Detection

Error detection is paramount in computer networks because it helps maintain data integrity and ensures that the information received is an accurate representation of what was sent. Without proper error detection mechanisms, data corruption can lead to serious consequences, including financial losses, compromised security, and degraded user experiences. By implementing effective error detection techniques, network administrators can minimize these risks and enhance the overall reliability of their systems.

Different Types of Error Detection Techniques

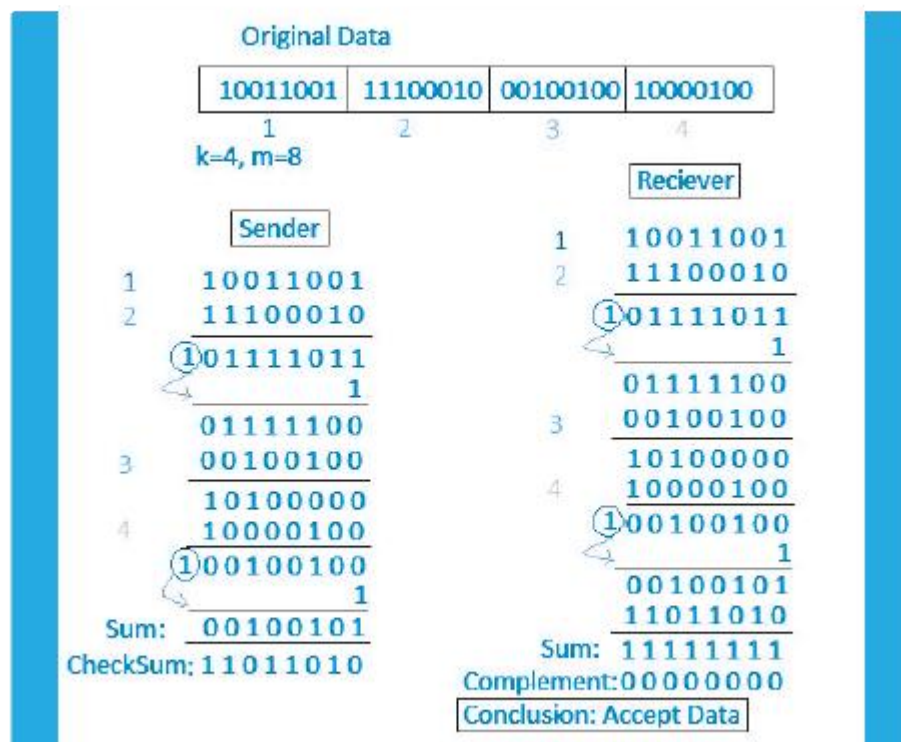
Several error detection techniques have been developed and standardized to safeguard data during transmission. Here are a few prominent ones:

1. Parity Check: Parity bits are added to data to make the total number of 1s either even or odd, thereby allowing the detection of single-bit errors. While simple and easy to implement, parity checks are limited in their ability to detect multiple errors.

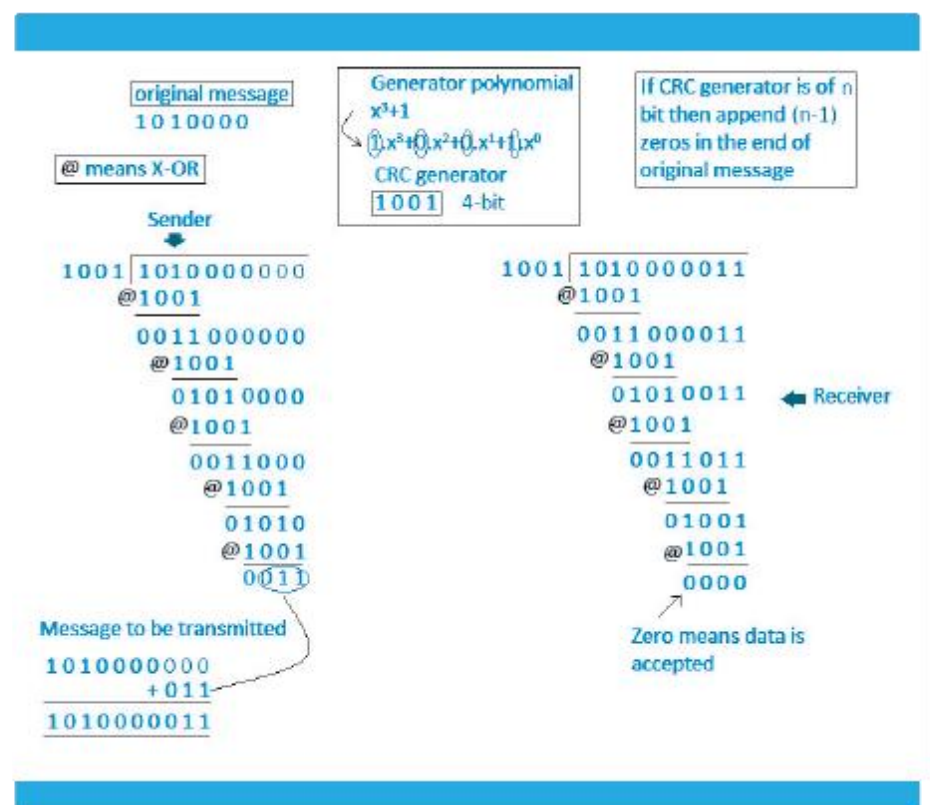
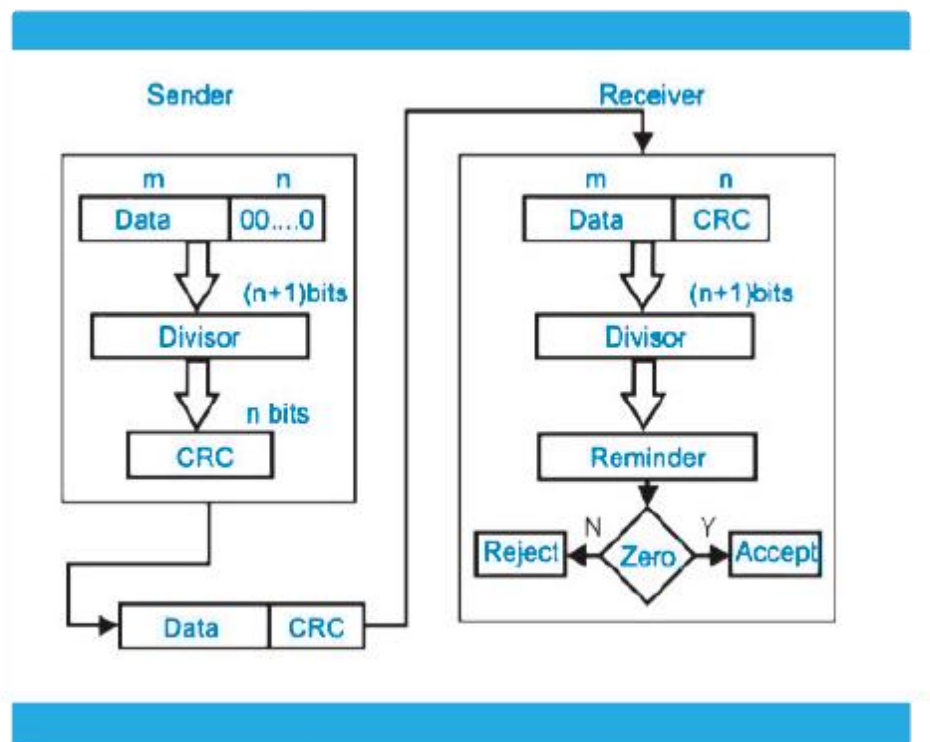


2. Checksums: Checksums involve the calculation of a sum or hash value from the data bits and appending it to the data being transmitted. The recipient performs the same calculation

and compares the results. Any mismatch indicates an error.



3. Cyclic Redundancy Check (CRC): CRC is a widely used error detection technique that involves polynomial division. The sender and receiver both use the same polynomial to perform division, and any remainder indicates the presence of errors.



Error Correction

Error Correction codes are used to detect and correct the errors when data is transmitted from the sender to the receiver.

Error Correction can be handled in two ways.

- **Backward error correction:** Once the error is discovered, the receiver requests the sender to retransmit the entire data unit.
- **Forward error correction:** In this case, the receiver uses the error-correcting code which automatically corrects the errors.

A single additional bit can detect the error, but cannot correct it.

For correcting the errors, one has to know the exact position of the error. For example, If we want to calculate a single-bit error, the error correction code will determine which one of seven bits is in error. To achieve this, we have to add some additional redundant bits.

Suppose r is the number of redundant bits and d is the total number of the data bits. The number of redundant bits r can be calculated by using the formula:

$$2^r \geq d + r + 1$$

The value of r is calculated by using the above formula. For example, if the value of d is 4, then the possible smallest value that satisfies the above relation would be 3

To determine the position of the bit which is in error, a technique developed by R.W Hamming is Hamming code which can be applied to any length of the data unit and uses the relationship between data units and redundant units. So first we will discuss step wise

What Is Hamming Code?

The Hamming Code method is a network technique designed by R.W.Hamming, for damage and error detection during data transmission between multiple network channels.

The Hamming Code method is one of the most effective ways to detect single-data bit errors in the original data at the receiver end. It is not only used for error detection but is also for correcting errors in the data bit.

Let's look into some important terms related to the hamming code.

Important Terms for Hamming Code

To begin with, the steps involved in the detection and correction of data using hamming code, we need to understand some important terms and expressions, which are:

1. **Redundant Bits** - These are the extra binary bits added externally into the original data bit to prevent damage to the transmitted data and are also needed to recover the original data.

The expression applied to deduce the redundant value is,

$$2^r \geq d+r+1$$

Where,

d - "Data Bits"

r - "Redundant Bits", $r = \{1, 2, 3, \dots, n\}$

Example: Assuming the number of data bits is 7, find the number of redundant bits.

- $2^r \geq r+7+1$
- $2^4 \geq 4+8$ [$r=4$]

The number of redundant bits = 4.

2. Parity Bits - The parity bit is the method to append binary bits to ensure that the total count of 1's in the original data is even bit or odd. It is also applied to detect errors on the receiver side and correct them.

Types of parity bits:

- Odd Parity bits - In this parity type, the total number of 1's in the data bit should be odd in count, then the parity value is 0, and the value is 1.
- Even Parity bits - In this parity type, the total number of 1's in the data bit should be even in count; then the parity value is 0, and the value is 1.

Now we will look into step by step working of the hamming code.

Working of Hamming Code

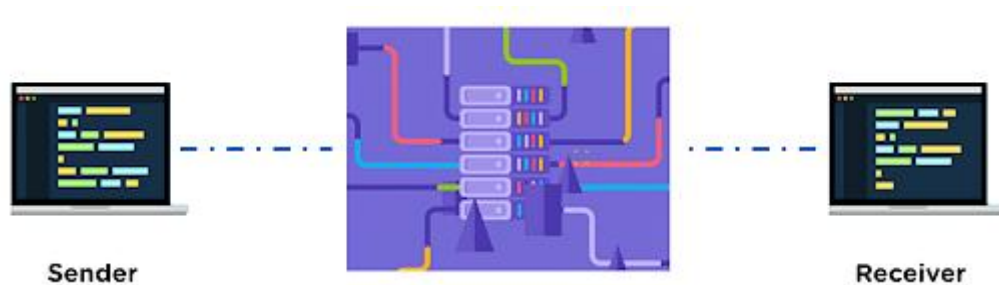
To solve the data bit issue with the hamming code method, some steps need to be followed:

- Step 1 - The position of the data bits and the number of redundant bits in the original data. The number of redundant bits is deduced from the expression $[2^r \geq d+r+1]$.
- Step 2 - Fill in the data bits and redundant bit, and find the parity bit value using the expression $[2^p]$, where, $p = \{0, 1, 2, \dots, n\}$.

- Step 3 - Fill the parity bit obtained in the original data and transmit the data to the receiver side.
- Step 4 - Check the received data using the parity bit and detect any error in the data, and in case damage is present, use the parity bit value to correct the error.

Next, we will solve an example using hamming code to clarify any doubts regarding the working steps.

Example for Hamming Code



To better understand the working of the hamming code, the following example is to be solved:

The data bit to be transmitted is 1011010, to be solved using the hamming code method.

- **Determining the Number of Redundant Bits and Position in the Data,**

The data bits = 7

The redundant bit,

- $2^r \geq d+r+1$
- $2^4 \geq 7+4+1$
- $16 \geq 12$, [So, the value of $r = 4$.]

Position of the redundant bit, applying the 2^p expression:

- $2^0 - P1$
- $2^1 - P2$
- $2^2 - P4$
- $2^3 - P8$

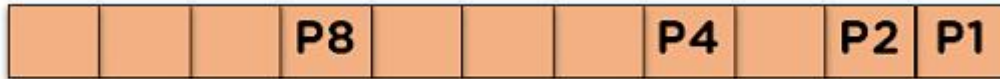


Figure 1

Applying the data bits in Fig. 1.



Figure 2

- **Finding the Parity Bits, for "Even parity bits,"**

1. P1 parity bit is deduced by checking all the bits with 1's in the least significant location.

P1: 1, 3, 5, 7, 9, 11

- P1 - P1, 0, 1, 1, 1, 1
- P1 - 0

2. P2 parity bit is deduced by checking all the bits with 1's in the second significant location.

P2: 2, 3, 6, 7, 10, 11

- P2 - P2, 0, 0, 1, 0, 1
- P2 - 0

3. P4 parity bit is deduced by checking all the bits with 1's in the third significant location.

P4: 4, 5, 6, 7

- P4 - P4, 1, 0, 1
- P4 - 0

4. P8 parity bit is deduced by checking all the bits with 1's in the fourth significant location.

P8: 8, 9, 10, 11

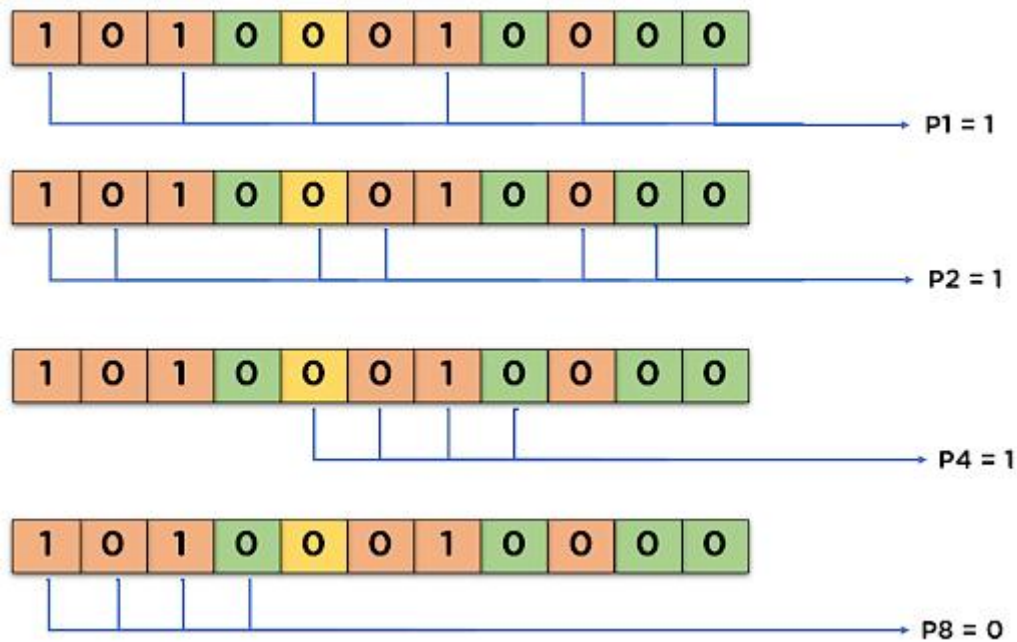
- $P8 - P1, 1, 0, 1$
- $P8 - 0$

So, the original data to be transmitted to the receiver side is:

11	10	9	8	7	6	5	4	3	2	1
1	0	1	0	1	0	1	0	0	0	0

Error Detecting and Correction of the Data Received,

Assume that during transmission, the data bit at position 7 is changed from 1 to 0. Then by applying the parity bit technique, we can identify the error:



Parity values obtained in the above deduction vary from the originally deduced parity values, proving that an error occurred during data transmission.

To identify the position of the error bit, use the new parity values as,

- $[0+2^2+2^1+2^0]$
- 7, i.e., same as the assumed error position.

To correct the error, simply reverse the error bit to its complement, i.e., for this case, change 0 to 1, to obtain the original data bit.

With the completion of the example for hamming code, we can conclude this document on “Hamming Code for Error Detection and Correction.”

Cyclic Redundancy Check (CRC)

It is another error detection technique to detect errors in data that has been transmitted on a communications link. A sending device applies a 16 or 32 bit polynomial to a block of data that is to be transmitted and appends the resulting cyclic redundancy check (CRC) to the block. The receiving end applies the same polynomial to the data and compares its result with the result appended by the sender. If they agree, the data has been received successfully. If not, the sender can be notified to resend the block of data.

At Sender Side

- Sender has a generator $G(x)$ polynomial.
- Sender appends $(n-1)$ zero bits to the data.
Where n = no of bits in generator
- Dividend appends the data with generator $G(x)$ using modulo 2 division (arithmetic).
- Remainder of $(n-1)$ bits will be CRC.

Codeword: It is combined form of Data bits and CRC bits i.e. Codeword = Data bits + CRC bits

Example

Assume that –

(a) data is 10110.

(b) code generator is 1101. (Code generator can also be mentioned in polynomial : x^3+x^2+1)

Note – Code generator is always mentioned in the question.

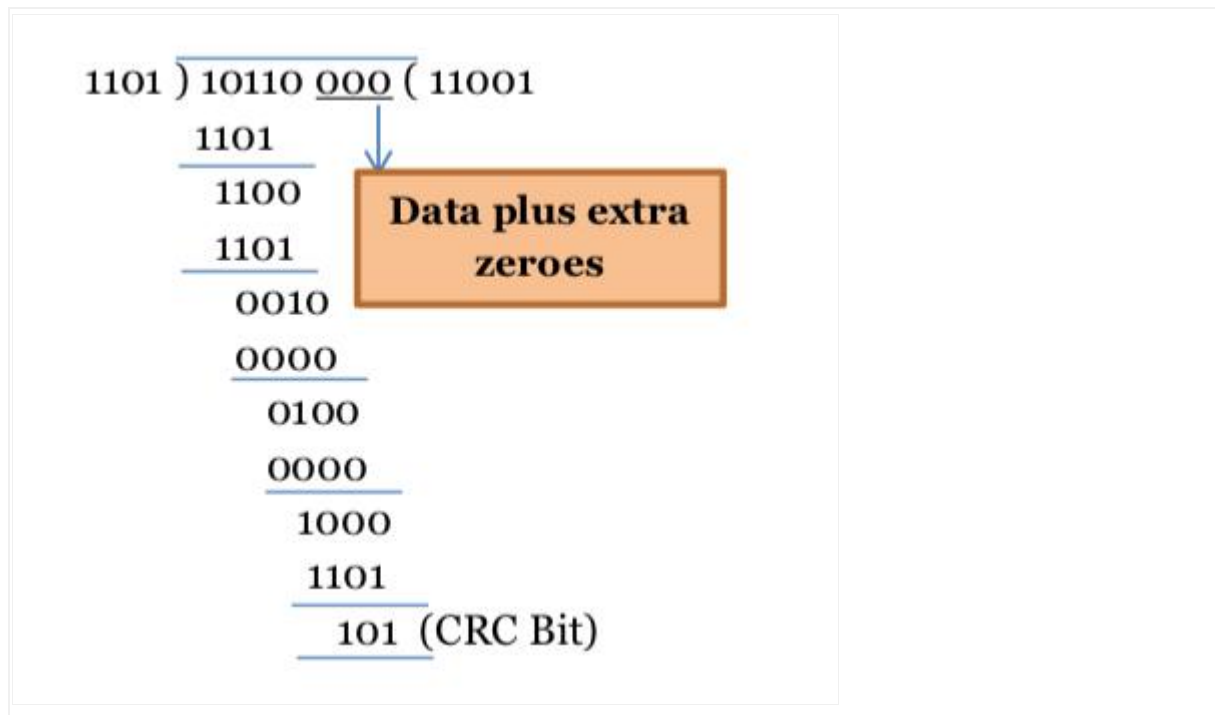
Calculate CRC Bits:

Now, we will see how to calculate CRC bits using above data.

While calculating the CRC bits, we pad $(n-1)$ 0's to the message bits, where 'n' = no of bits in the code generator.

In the above code, code generator is 1101. So, there is total 4 bits. So, we will append 000 with the data.

Finally, Cyclic Redundancy check will be generated as shown below –



CRC calculation by sender

Cyclic redundancy check is 101. Thus, the sender sends 10110101 to the receiver.

At Receiver Side

- Receiver has same generator $G(x)$.
- Receiver divides received data (data + CRC) with generator.
- If remainder is zero, data is correctly received.
- Else, there is error.

Assume the received message is 10110110.

Calculate CRC Bits:

It does not add any padding bits, rather calculates from the entire received code word.

$$\begin{array}{r}
 1101 \overline{) 10110110} \quad (11001 \\
 \underline{1101} \\
 1100 \\
 \underline{1101} \\
 0011 \\
 \underline{0000} \\
 0110 \\
 \underline{0000} \\
 1100 \\
 \underline{1101} \\
 \underline{001} \text{ (CRC Bit)}
 \end{array}$$

CRC calculation by receiver

The CRC bits are calculated to be different. Thus, there is an error detected.

CRC Rules:

- CRC generator should not contain 'x'.
- If (x+1) is generator, it can detect odd no of error.
- If generator is x^n , then last n bit errors can be detected.

Performance of Cyclic Redundancy Check

CRC is a very effective error detection method. If the divisor is chosen according to the above mentioned rules,

- CRC can detect all burst errors that affect an odd number of bits.
- CRC can detect all burst errors of length less than or equal to the degree of the polynomial.
- CRC can detect with a very high probability, burst errors of length greater than the degree of the polynomial.

Thus, we are now acquainted with all the error detection and correction techniques. We have read about the hamming code, parity check, Checksum method and cyclic redundancy check method. In the next tutorials, we will study about the various switching methods used for communication and data transmission in computer networks.